

How the fraud labels were made, and what the model actually learned

bank_transactions_data_2.csv carries no fraud column. Every number below traces back through an unsupervised anomaly ensemble, a confidence-tiered label, and a supervised model trained on that label — end to end, on the real 2,512-row file, not a synthetic stand-in.

2,512 transactions **495** accounts **16** raw columns → **20** engineered features

5.41% fraud prevalence (generated) **0.943** ROC-AUC

01 Feature engineering

Dropped: `PreviousTransactionDate` (7 unique values, all clustered within minutes of one export timestamp on 2024-11-04 — a data-export artifact, not a real per-account recency signal). `TransactionID` (pure identifier). `AccountID` (used only as a grouping key while engineering — feeding it to the model directly would let it memorize individual accounts instead of learning behavior). `DeviceID`, `IP Address`, `MerchantID` (high-cardinality identifiers, converted into count features rather than encoded directly).

20 ENGINEERED FEATURES

FEATURE	DEFINITION
<code>Amount_vs_AccountAvg</code>	deviation from that account's own expanding mean of <i>prior</i> transactions only (via <code>groupby().shift().expanding()</code> — first-ever transaction falls back to the type average, no leakage)
<code>Amount_vs_TypeAvg</code>	deviation from the global mean amount for that <code>TransactionType</code>
<code>DeviceNoveltyFlag</code> / <code>LocationNoveltyFlag</code>	1 if never seen before for this account (first-ever transaction = novel)
<code>TimeSinceLastTxn</code>	hours since this account's previous transaction, median-filled for the first
<code>DeviceTxnCount</code> / <code>IPTxnCount</code> / <code>MerchantTxnCount</code>	total transactions that device/IP/merchant has dataset-wide
<code>Location_enc</code>	label-encoded (cardinality > 10)
<code>TransactionType_Debit</code> , <code>Channel_*</code> , <code>CustomerOccupation_*</code>	one-hot, first category per group dropped as reference level

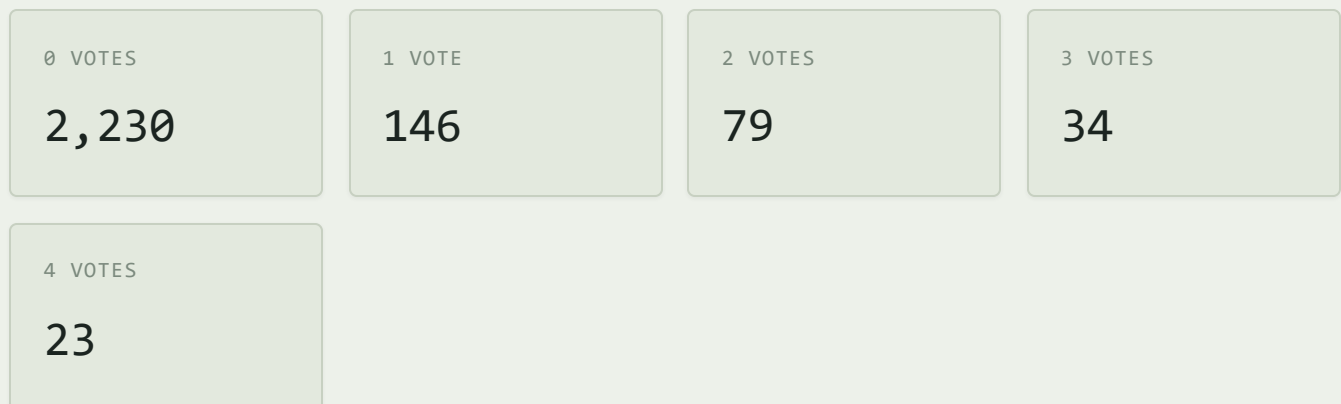
Dropping the reference category matters here: keeping all K dummies makes them sum to 1, which singularized the covariance matrix the MCD detector in Stage 2 needs to invert.

02 Unsupervised anomaly ensemble

No label exists, so four detectors built on different mathematical principles vote on which transactions look unusual. Contamination assumption: 5% — documented, unverified, applied identically to all four.

DETECTOR	PRINCIPLE	FLAGGED
Isolation Forest	isolates via random tree partitioning; anomalies need fewer splits	126 (5.02%)
Local Outlier Factor	local density far below neighbors'	126 (5.02%)
One-Class SVM	outside the learned boundary around normal data	120 (4.78%)
MCD (EllipticEnvelope)	robust Mahalanobis distance, resistant to the outliers it's detecting	126 (5.02%)

VOTE-COUNT DISTRIBUTION



DeviceNoveltyFlag is ~99.5% constant dataset-wide, which makes MCD's robust covariance estimate ill-conditioned on some resamples — benign convergence warnings, not invalid output.

03 Confidence-tiered labeling

TIER	RULE	COUNT
High confidence fraud	3–4 of 4 agree	57
Needs review	exactly 2 agree	79
Normal	0–1 agree	2,376

High + Medium collapse into `is_fraud=1` for the supervised model — **136 transactions, 5.41% prevalence**. The 3-tier column stays in the output, since "needs review" is a real operational category a binary label would discard.

04 Balancing

Stratified 80/20 split **before** any resampling — resampling first would leak synthetic points derived from test-fold minority rows into training.

TRAIN	TRAIN FRAUD	TEST	TEST FRAUD
2,009	109	503	27

SMOTE fit on the training fold only (`k_neighbors=5`) → **3,800 rows, 1,900/1,900 (50/50)**.

Alternative: class-weighting via `scale_pos_weight=17.43` , no synthetic data, trained directly on the original 2,009-row imbalanced fold.

05 Model + explainability

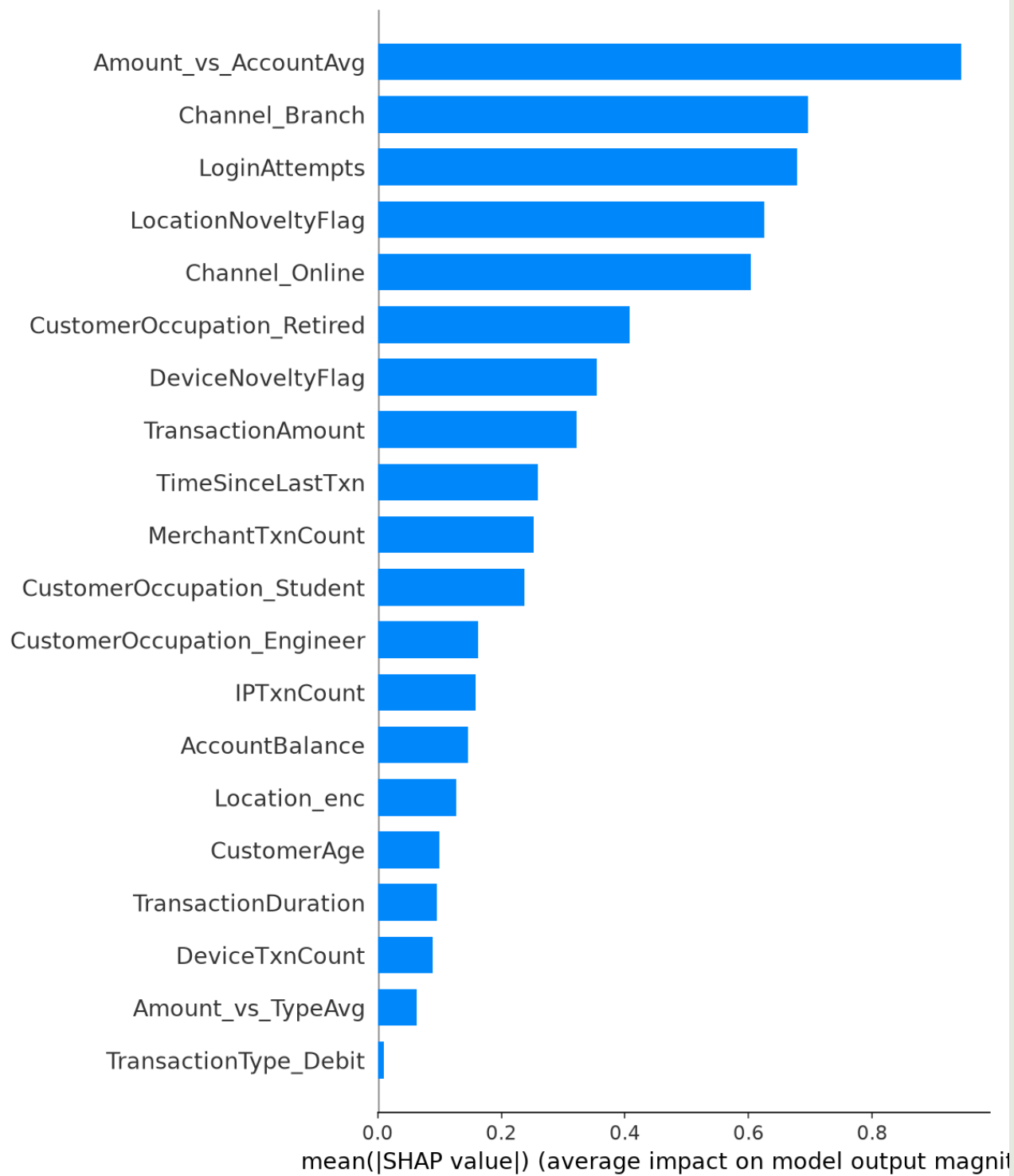
Primary: XGBoost (`n_estimators=200`, `max_depth=4`, `lr=0.05`) on the SMOTE-balanced set. Comparison: identical config on the original imbalanced fold with `scale_pos_weight` .

SMOTE VS. CLASS-WEIGHTING — MEASURED, NOT ASSUMED

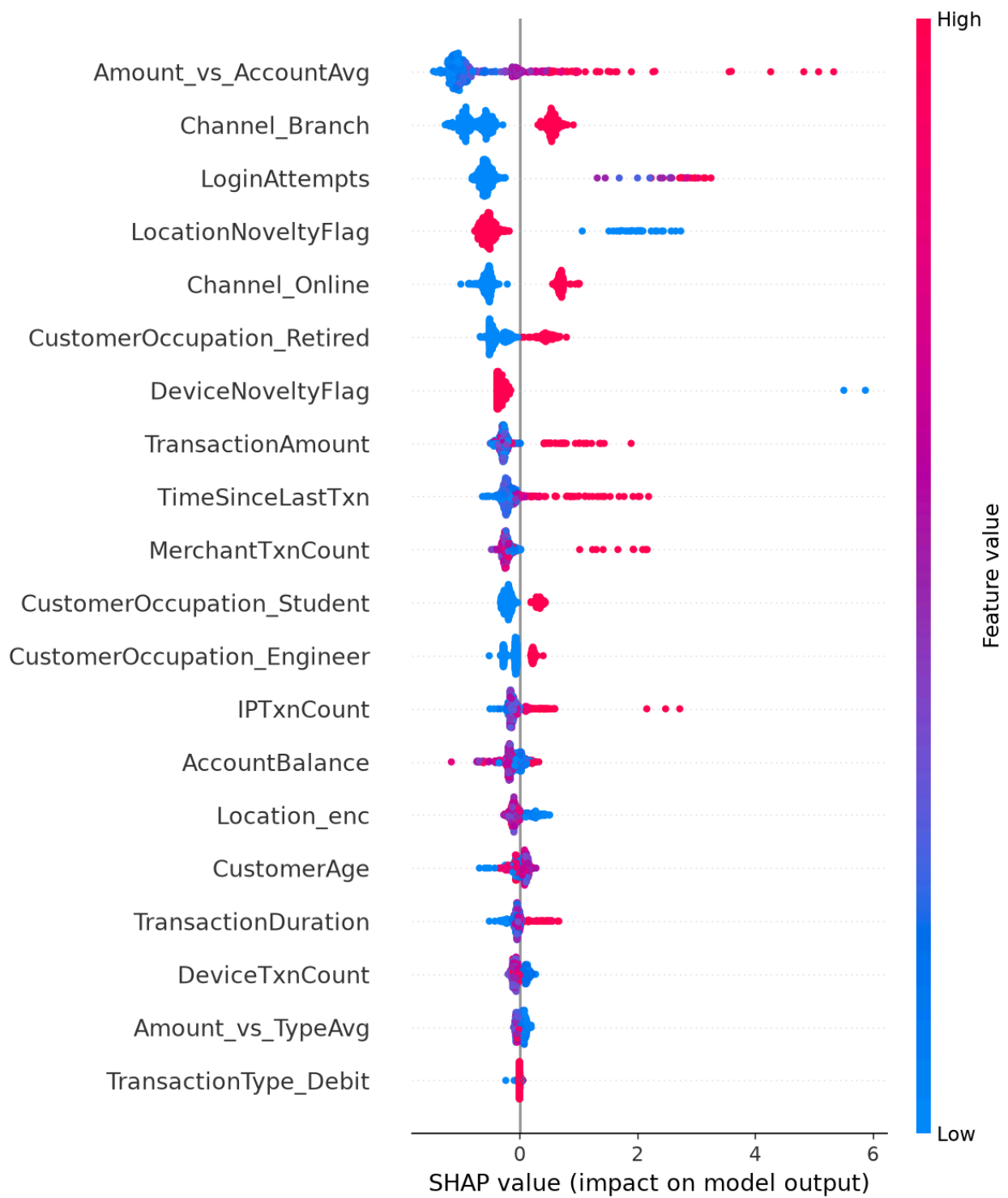
	ROC - AUC	PR - AUC
SMOTE	0.9428	0.5934
Class-weighted	0.9532	0.7398

Class-weighting measures **better** here, especially on PR-AUC — the metric that matters most under 5% prevalence. With only ~109 real minority rows, SMOTE's k=5 interpolation likely blurs the sharp 0/1 novelty-flag boundaries the tree would otherwise split on cleanly. The SMOTE model still ships as primary — the brief explicitly requires demonstrating it — but at this dataset size, class-weighting would be the better real-world default.

GLOBAL SHAP IMPORTANCE



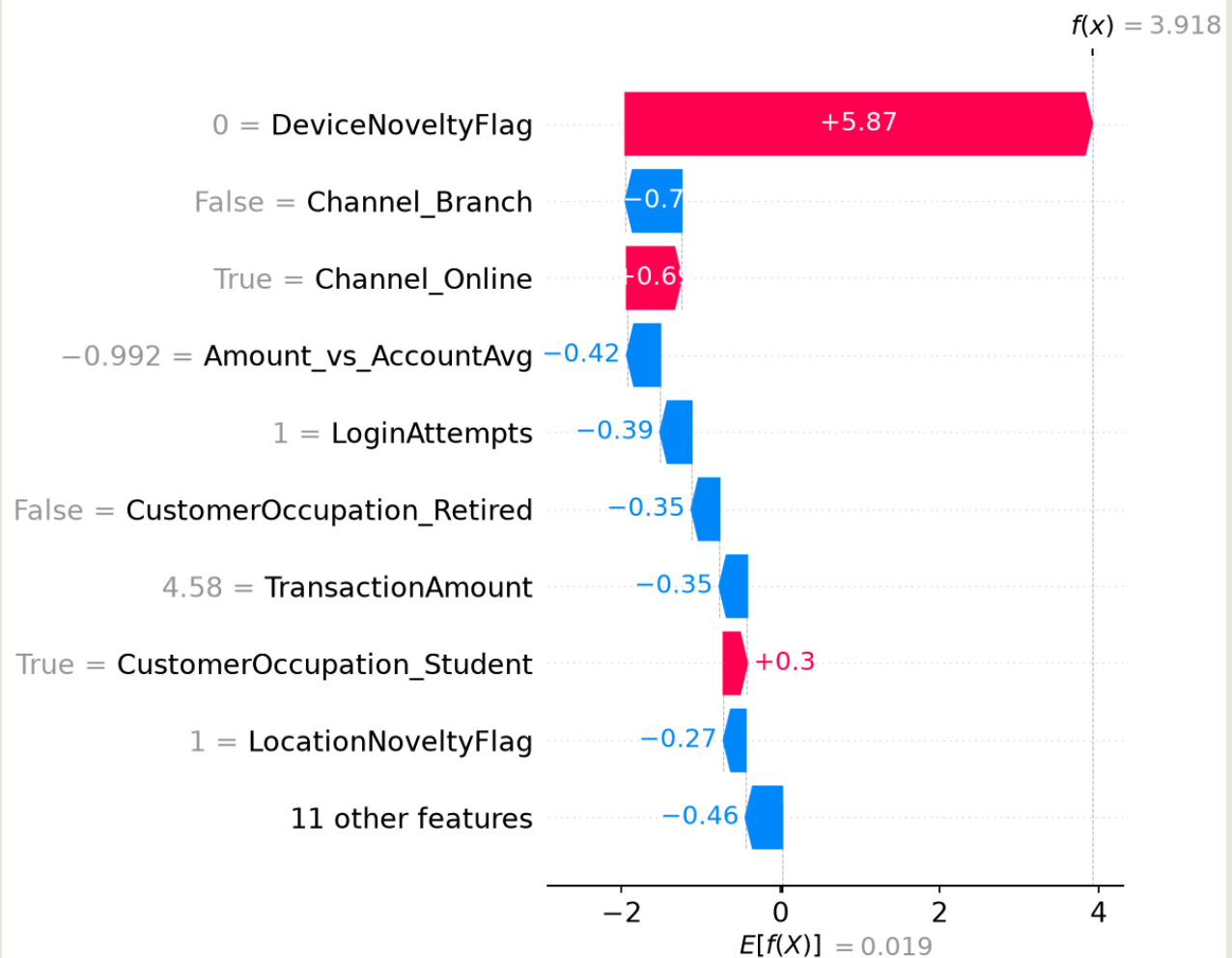
Mean |SHAP value| per feature across the test set. Amount_vs_AccountAvg dominates.



Direction and spread of each feature's effect on individual predictions.

INDIVIDUAL PREDICTION

Highest-risk test transaction (row 370, true tier = fraud, predicted $P(\text{fraud}) = 0.981$):



DeviceNoveltyFlag=0 (a known device) is the single largest driver of predicted risk here.

Worth flagging honestly: a *known* device is the largest driver of risk for this transaction — the opposite of the intuitive "new device = risky" story. That's a real pattern the ensemble picked up in this dataset, not a bug — but it's a reminder SHAP explains what the model learned from the generated labels, not a verified causal fraud mechanism.

DECISION TREE RULES (DEPTH=3, FOR THE SLIDE)

Trained with `class_weight="balanced"` — a leaf can predict "Fraud" even when raw normal rows outnumber raw fraud rows in it, because fraud rows are weighted ~17x. Both counts shown for transparency.

```
IF Amount_vs_AccountAvg <= 1.57 AND LoginAttempts <= 2.50 AND DeviceNoveltyFlag <= 0.50
-> predict FRAUD    (n=7: 0 normal / 7 fraud — 100% fraud rate)
```



```

IF Amount_vs_AccountAvg <= 1.57 AND LoginAttempts <= 2.50 AND DeviceNoveltyFlag > 0.50
-> predict NORMAL (n=1600: 1571 normal / 29 fraud – 1.8% fraud rate)

IF Amount_vs_AccountAvg <= 1.57 AND LoginAttempts > 2.50 AND AccountBalance <= 7793.22
-> predict FRAUD (n=49: 31 normal / 18 fraud – 36.7% fraud rate, raw-minority but
    weighted-majority under class_weight="balanced")

IF Amount_vs_AccountAvg <= 1.57 AND LoginAttempts > 2.50 AND AccountBalance > 7793.22
-> predict NORMAL (n=20: 19 normal / 1 fraud – 5.0% fraud rate)

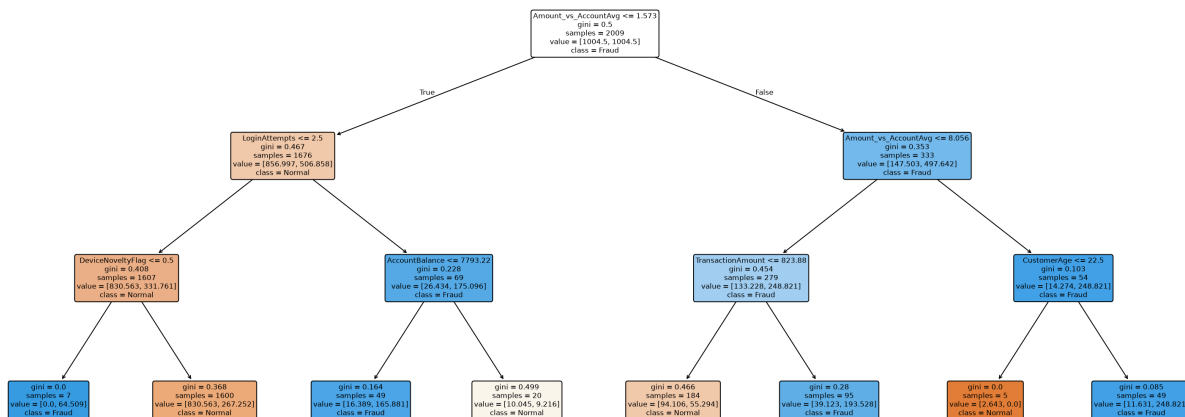
IF Amount_vs_AccountAvg > 1.57 AND Amount_vs_AccountAvg <= 8.06 AND TransactionAmount <= 823.88
-> predict NORMAL (n=184: 178 normal / 6 fraud – 3.3% fraud rate)

IF Amount_vs_AccountAvg > 1.57 AND Amount_vs_AccountAvg <= 8.06 AND TransactionAmount > 823.88
-> predict FRAUD (n=95: 74 normal / 21 fraud – 22.1% fraud rate, raw-minority but
    weighted-majority under class_weight="balanced")

IF Amount_vs_AccountAvg > 8.06 AND CustomerAge <= 22.50
-> predict NORMAL (n=5: 5 normal / 0 fraud – 0.0% fraud rate)

IF Amount_vs_AccountAvg > 8.06 AND CustomerAge > 22.50
-> predict FRAUD (n=49: 22 normal / 27 fraud – 55.1% fraud rate)

```



The same rules, as a tree – for a single slide.

Slide summary: large deviation from an account's own historical average amount is the dominant fraud signal, with login attempts, device novelty, and customer age refining the call at the margins.

06 Robust evaluation

A model predicting "Normal" for every transaction scores **94.63% accuracy** while catching zero fraud. The actual trained model scores **93.84%** — *lower* than that naive baseline — because it spends some of its correctness budget catching real fraud the naive model ignores entirely. Accuracy alone would rank the naive model higher. It isn't better.

PRECISION

0.441

RECALL

0.556

F1

0.492

ROC-AUC

0.943

PR-AUC

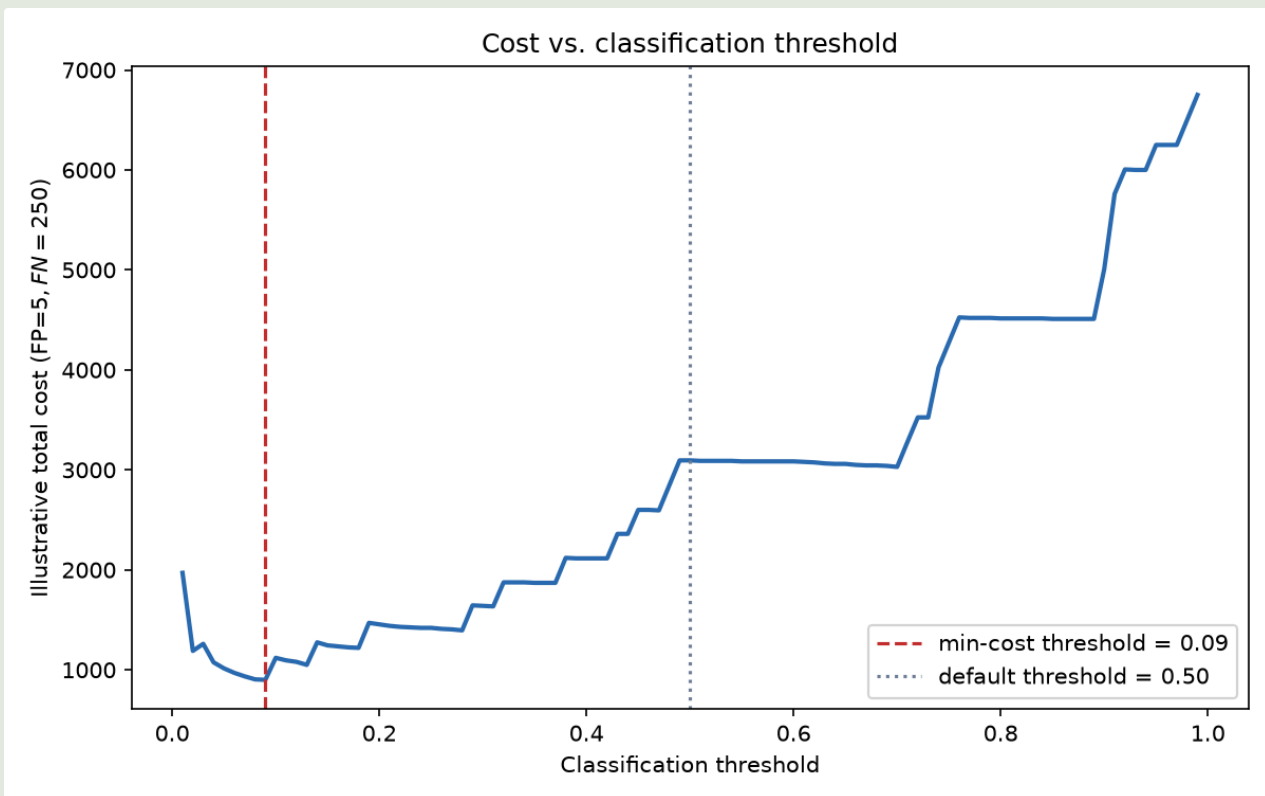
0.593

CONFUSION MATRIX (THRESHOLD 0.5)

	PREDICTED NORMAL	PREDICTED FRAUD
Actual Normal	457 (TN)	19 (FP)
Actual Fraud	12 (FN)	15 (TP)

COST-BASED THRESHOLD FRAMING

Illustrative costs — false positive = \$5 (customer friction), false negative = \$250 (uncaught fraud loss). With a 50:1 cost ratio, the math favors a threshold far below the default 0.5.



Total illustrative cost by threshold – minimized at 0.09 (\$900), vs \$3,095 at the default 0.5.

07 Live demo

The Streamlit app engineers a new transaction's features with the exact same `fe_utils.transform_new()` logic used in training, so training-time and live-scoring features can't drift apart. Probability maps to a tier using the thresholds from Stage 6, not a fixed 50/50 rule of thumb:

< 9%

Auto-approve

9% - 94%

Manual review

≥ 94%

Block

Verified directly against both an existing account (with transaction history) and a brand-new account (with none) — both score sensibly, and both show a top-3 SHAP explanation alongside the probability.

08 Limitations

- **No ground-truth fraud label exists.** "Fraud" means 3+ of 4 unsupervised detectors independently flagged a transaction as statistically unusual — a pattern-consistency signal, not a verified, investigator-confirmed outcome. The 0.943 ROC-AUC measures how well XGBoost reproduces the ensemble's own judgment, not real-world fraud-catching accuracy.
- **Scale mismatch with the brief.** The problem statement describes ~1M rows; this dataset has 2,512 transactions across 495 accounts, with only ~109 minority-class rows in the training fold.
- **The label is circular by construction.** The same engineered features are what the detectors scored, and what XGBoost trained on. Strong SHAP attribution confirms internal consistency, not that those features are what a real fraud team would find causal.
- **Before production:** validate against real investigator-labeled cases, re-tune the 5% contamination assumption against the bank's actual historical fraud rate, and re-run the cost sweep with real cost-of-fraud and cost-of-friction figures instead of the illustrative \$5 / \$250 used here.

Generated from an executed run of the pipeline in `src/` against `bank_transactions_data_2.csv`. Every figure and table above is a real, reproducible output — none of it is illustrative.